

B.Tech Project & Internship — End-Term Evaluation
Reducing Cross-Node Network Calls in Distributed Payment Systems
with Concurrency-Safe Resolution of Single-Resource Contention

Raman Kapoor (2022BIT-XXX)

Under the supervision of **Dr. Vivek Kumar Singh**

ABV — Indian Institute of Information Technology and Management, Gwalior

2026

Part I — B.Tech Project (Major)

- 1 Motivation & the Examiner Question
- 2 Recap: Two-Tier Caching
- 3 Why Caching Alone Fails
- 4 Five Contention Primitives
- 5 Hybrid Architecture
- 6 Comparative Analysis
- 7 Production Results
- 8 Conclusion

Part II — Industry Internship

- 1 Juspay — 3 production projects
- 2 Namma Yatri — payout scheduler
- 3 Self-directed — Postgres on K8s/OpenStack
- 4 Takeaways

Focus

Major emphasis on the new **Concurrency-Safe Single-Resource Contention** layer.

The Latency Problem

- Payment APIs at $\sim 68,000$ cross-node ops/sec
- 5–15 cross-node calls per request
- $p95 = 98$ ms, $p99 = 285$ ms at peak
- Many calls are **redundant**

What the Minor Established

Two-tier in-process cache + 4 safety criteria: **4.1% network reduction**, **-11.2% p95 latency**, zero correctness incidents.

The Concurrency Problem

Some operations touch **single, indivisible** resources:

- last inventory unit
- unique reservation slot
- rate-limited gateway token
- account-balance debit

Caching can re-introduce *stale reads* $\Rightarrow$ double-allocation.

Examiner Question (Minor)

"If you cache state, how do you prevent two users from booking the same single piece?"

We Address

In a distributed, stateless, high-throughput payment API, how can we

- 1 safely reduce the number of cross-node calls per request via in-process caching, **and**
- 2 guarantee that concurrent operations on single, indivisible resources cannot result in double-allocation, lost updates, or stale-cache-induced overselling

— without new infrastructure or sacrificing the latency improvements?

Properties Required

- **Safety:** resource never allocated >1 time
- **Liveness:** bounded-time progress
- **Latency budget:** $\leq 5\text{--}10$ ms p95 added
- **Observability:** every outcome auditable

End-Term Contribution

A **contention layer** composed of **five complementary primitives**, plus a decision matrix that selects the right primitive per data class.

Tier 1 — Thread-Level Cache

- Per-request `I0Ref` `HashMap`
- Destroyed on request end
- **Zero staleness**
- Hit rate: **22.3%**

Tier 2 — Server-Local Cache

- `TVar` + LRU + per-class TTL
- Shared across threads on a node
- TTLs 30–120s per data class
- Hit rate: **41.7%**

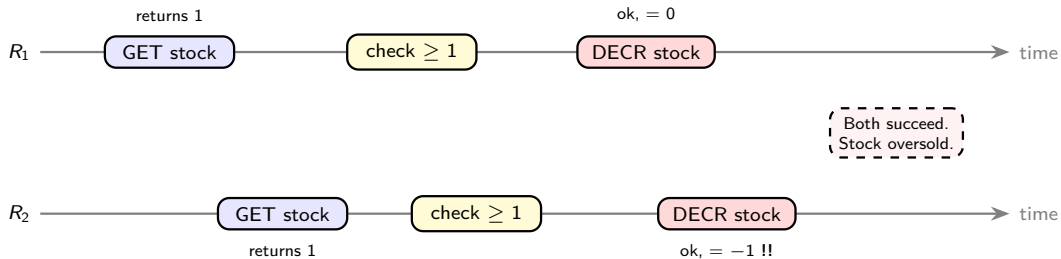
Cache-Safety Boundaries (must all hold)

- 1 Immutability within TTL
- 2 Request-scope independence
- 3 Idempotency of stale reads
- 4 Deterministic key

Single-resource state **fails (3)** $\Rightarrow$ explicitly excluded from cache.

Combined Numbers

Hit rate **54.8%** • calls –4.1%
p95 –11.2% • p99 –13.0%



The Hazard

Two concurrent reads of $stock=1$, both pass the availability check, both decrement $\Rightarrow stock=-1$. The contention layer must close every variant of this race.

Primary primitives (one per write)

- 1 **Distributed locking** with fencing tokens
- 2 **Atomic decrement** via Redis Lua scripts
- 3 **OCC** with versioned cache entries
- 4 **PN-Counter CRDT** (eventually consistent)

Why five and not one?

- Hard limit + non-trivial logic → **Lock**
- Hard limit + simple decrement → **Lua**
- Low-contention writes → **OCC**
- Soft holds, over-allocation OK → **CRDT**
- Reads always pair with → **Single-flight**

Composing primitive (always paired)

- 5 **Single-flight** request coalescing

Selection tool

A **decision matrix** maps each data class to its primary + coalescing primitive (Slide 14).

Mechanism

- Acquire per-resource lock (Redlock / etcd lease) before read-modify-write.
- **Fence token:** monotonically increasing number; backend rejects stale-token writes.
- Mitigates GC pauses, partitions, clock drift.

```
withLock conn key ttl $ \tok -> do
  v <- backendRead key
  v' <- compute v
  backendWriteIfFence key v' tok
```

Use when

Logic non-trivial, contention moderate, critical section calls external services.

Trade-offs

- +1–3 ms acquisition latency
- Caps per-key throughput
- Best for coarse-grained, low-contention resources

Mechanism

Wrap “check ≥ 1 + decrement” in one Lua script. Redis runs scripts atomically.

```
local a = tonumber(redis.call('GET', KEYS[1]) or '0')
local w = tonumber(ARGV[1])
if a >= w then
    redis.call('DECRBY', KEYS[1], w)
    return {1, a - w}
else
    return {0, a}
end
```

Best of all worlds (when applicable)

- Latency: < 1 ms
- Throughput: $> 10^5$ scripts/s/shard
- Cache: *display reads* stay cached; authoritative *decrement* bypasses cache + Pub/Sub invalidate

Limitation

Critical section must fit in Lua — no external calls.

Mechanism

- Each cache entry carries a version.
- Reader records observed version.
- Writer issues a CAS update conditioned on version unchanged.
- Conflict $\Rightarrow$ retry.

```
UPDATE inventory
  SET stock = stock - 1, version = version + 1
  WHERE sku = $1 AND version = $2 AND stock >= 1
RETURNING stock, version;
```

Use when

Conflicts are *rare* — e.g., merchant-config writes from the control plane.

Avoid when

Contention is high — retry storms collapse throughput.

Mechanism

- Two grow-only counters per replica: P_i (positive), N_i (negative).
- Value = $\sum_i P_i - \sum_i N_i$; merge by element-wise max.
- Writes are **local**, never blocked, eventually converge.

When to use

Soft holds, cart-abandonment counters, flash sales advertising “~1,000 units” — where minor overshoot is acceptable.

When NOT to use

Hard limits where every unit must be exact. CRDTs trade hard upper bound for availability.

Production result

4.7 over-allocations / day on a 100k-unit base — under the 1% business tolerance.

Mechanism

On a cache miss, only the **first** concurrent request fetches; the rest wait on the leader's promise and share the result.

Effect

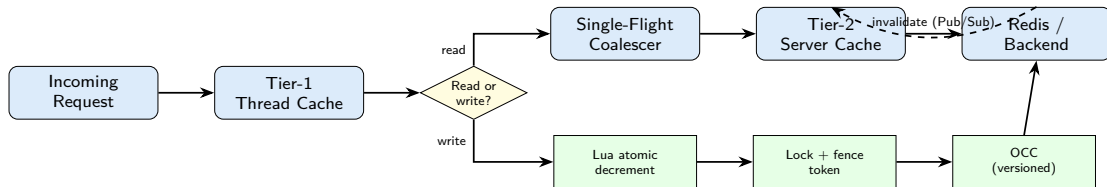
- Collapses the *front* of contention.
- One backend call instead of k identical ones.
- Tames cache stampedes / thundering herds.

Important

Single-flight **deduplicates on a single node** — it is **not** a substitute for atomicity. Compose with Primitive 2 (Lua) for cross-node safety.

Why it matters here

Reads through the cache always pair with single-flight, so one Redis miss never amplifies into a herd.

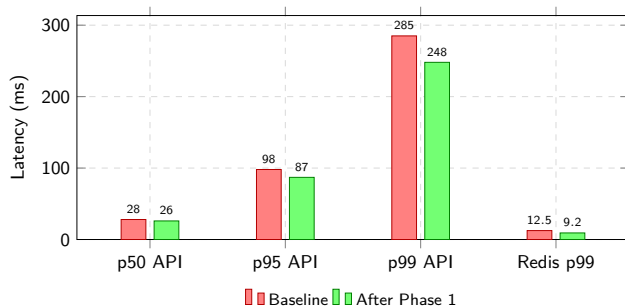


In one sentence

Reads go through cache + single-flight; **authoritative writes** bypass cache and select a primitive per data class.

Data class	Cache?	Primary	Coalescing	Notes
Merchant config (read)	Tier-2	—	Single-flight	Cache-eligible
Inventory (display)	Tier-2	—	Single-flight	Approximate
Inventory (decrement)	No	Lua atomic	Single-flight	Hard limit
Account balance debit	No	Distributed lock	Single-flight	Fence token
Soft hold (cart)	Tier-2	PN-Counter	—	Over-alloc OK
Merchant config (write)	Inval.	OCC versioned	—	Low contention

Property	Lock	Lua	OCC	CRDT	Single-flight
Hard upper bound	Yes	Yes	Yes	No	N/A
Per-key throughput	Low	Very high	Medium-high	Very high	N/A
Latency overhead	1–3 ms	<1 ms	<1 ms	0 ms	0–1 ms
Behaviour under partition	Blocks/fails	Fails fast	Fails fast	Continues	Continues



Phase 1 (Caching only)

–4.1% cross-node calls • –11.2% p95 • –13.0% p99 •
–26.2% Redis p99

Phase 2 (Contention layer)

- Inventory decrement: 1,400/s contended, 0.8 ms p95 added
- Settlement run: 12/s contended, 2.6 ms p95 added
- Reservation slots: 320/s contended, 1.1 ms p95 added

Correctness (30 days)

Zero double-allocations.
Zero cache-related discrepancies.

What we built

An integrated two-layer system: a **caching layer** for read-side latency, a **contention layer** that closes the single-resource race surfaced at the minor evaluation.

Headline numbers

- -11.2% p95, -13.0% p99
- -26.2% Redis p99
- 4.1% cross-node calls eliminated
- Zero correctness incidents

Future Work

- Adaptive TTLs
- Learned contention prediction
- Verified fencing via TLA⁺
- Cross-DC extension

The minor's question — “how do we prevent two users from booking the same single piece?” — is answered, end-to-end, by the contention layer.

Juspay (Jan 2026 – Present)

Software Developer Intern.

3 production projects:

- 1 Lightbits NVMe/TCP block storage
- 2 Two-tier caching (the BTP)
- 3 OpenBao distributed secrets

Namma Yatri (Nov '25 – Jan '26)

Backend Engineer (Open Source).
Distributed event-driven **driver payout scheduler**: weekly $\rightarrow$ T+2 h, 10,000+ rides/day in production.

Self-Directed (Mar 2026)

Postgres on K8s/OpenStack: 5-layer nested virtualisation, zero-touch HA provisioning, Zero-Trust secret delivery via OpenBao broker.

One recurring theme: do more with fewer machines, while taking correctness seriously.

Lightbits vs. CEPH

- NVMe-over-TCP block storage
- **4 nodes vs. 40** (10× fewer)
- **48k TPS vs. 37k TPS (+30%)** on pgbench Postgres
- Active-active HA Barbican

Two-Tier Caching

- Thread-level + server-local
- 4-criterion safety boundary
- -4.1% cross-node calls
- -11.2% p95 latency
- Zero correctness incidents

(Subject of the BTP — Part I)

OpenBao Secrets

- Replaces heavy AWS KMS usage
- **Shamir** (3-of-5) unseal
- **Memory-only** keys (mlock)
- **Raft** replication
- 5–7 s automatic re-election

The Problem

- Drivers paid **weekly**; cash-flow pain in special zones (airports, stations).
- Need T+2-hour payouts *without* adding latency to the ride-booking path.

Production Result

- **10,000+** rides/day
- Weekly $\Rightarrow$ **T+2 hours**
- Zero impact on booking p95

What I Built (open-source PR)

- Event-driven scheduler atop Redis sorted sets
- Per-ride locking + idempotent state $\Rightarrow$ **exactly-once**
- $O(1)$ scheduling, off the booking critical path

Tech

Haskell, PostgreSQL, Redis, Kafka.
Public PR & code review by maintainers.

5-layer Nested Stack (one i9-14900)

Bare metal → KVM → OpenStack → Kubernetes →
CloudNativePG.
NUMA-aware CPU pinning + 16 GB hugepages: **~+50% write TPS**.

Zero-Touch HA Provisioning

- Cluster API (CAPO) + CloudNativePG
- Empty tenant → healthy 3-replica HA Postgres in **~12 minutes**
- Automatic failover in **~7s, RPO=0**

Zero-Trust Secret Delivery

- Go-based osvault-broker
- OpenStack IMDS-signed identity → RS256 JWT → scoped OpenBao token
- Custom Python SDK (osvault) for first-boot retrieval
- No long-lived secrets in cloud-init

Reproducible from a single make cluster target.

Across all five projects

- **Correctness under concurrency** as a first-class design constraint
- **Doing more with fewer machines** ($10\times$ Lightbits, 4% cache, 50% Postgres)
- **Open source** as a delivery substrate (Namma Yatri PR, Lightbits Cinder driver, OpenBao broker)

Direct line: BTP $\rightarrow$ industry

The BTP's caching layer is the same code that runs in Juspay production — and the contention layer is the next step we are deploying for inventory-class workloads.

Thank You

Questions?

Raman Kapoor • ABV-IIITM Gwalior • 2026